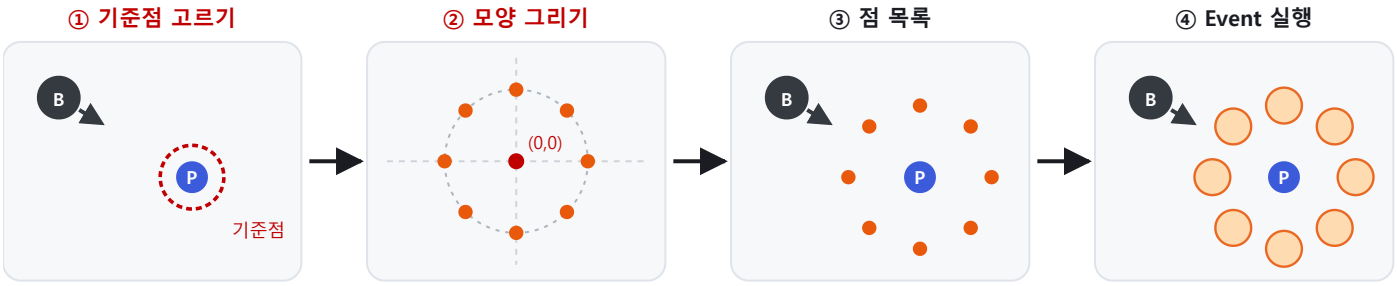
COMBATABILITY · 04 · 2026-10-30

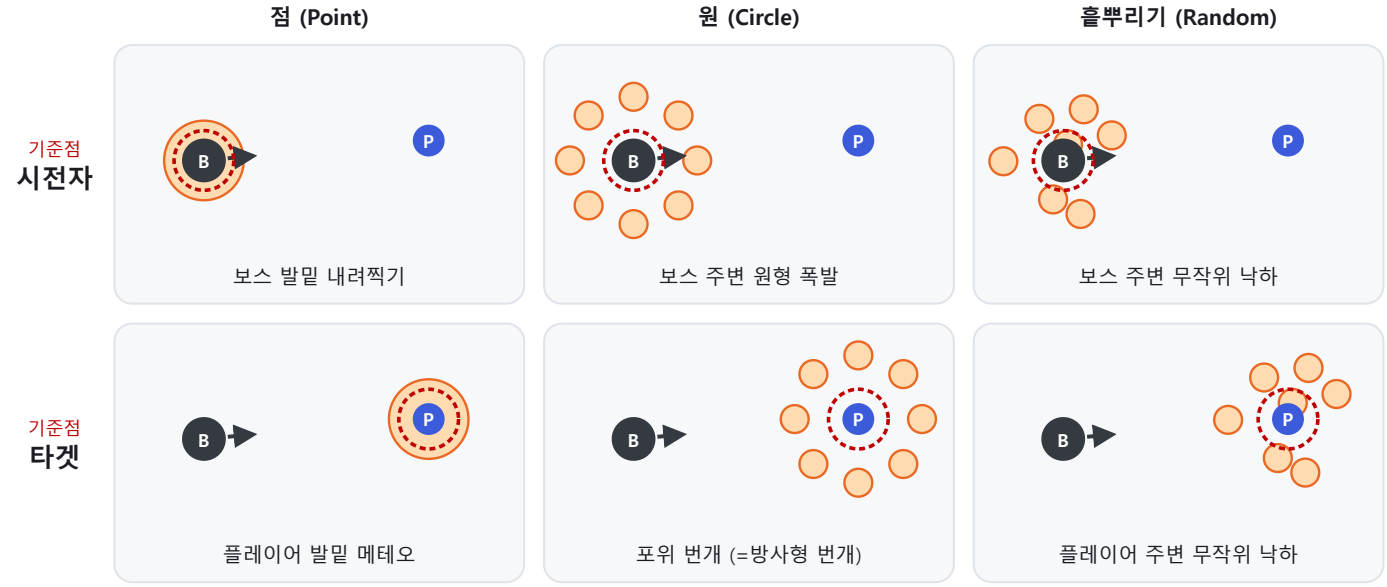
Placement — "어디에"를 만드는 부품

Event는 **무엇을**(때린다 · 소환한다), Placement는 **어디에**(점 목록)를 맡는다. Placement는 두 단계로 일한다: **기준점 하나 고르기 → 그 기준점 둘레에 모양 그리기.**

1. 한 번 실행될 때 일어나는 일



2. 기준점 2가지 × 모양 3가지 = 패턴 6가지

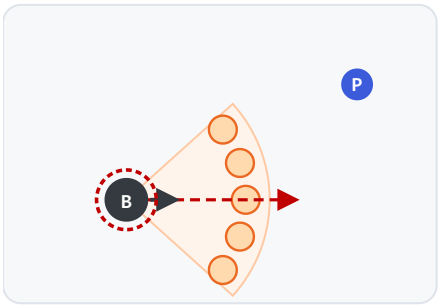


Event 코드는 StrikeEvent 하나 — 바뀌는 건 "기준점 + 모양" 에셋 조합뿐

3. 기준점은 점이 아니라 "위치 + 방향" (Pose)

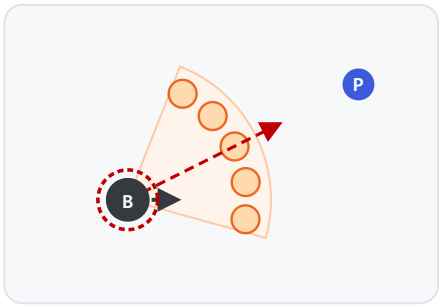
같은 부채꼴, 같은 보스, 같은 플레이어 — **방향 옵션 하나**만 다르다.

앵커 방향 그대로 (AnchorForward)



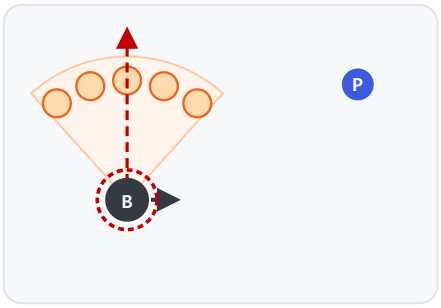
보스가 보는 쪽(→)으로 부채꼴

타겟 쪽으로 (TowardTarget)



보스 → 플레이어 쪽으로 부채꼴

월드 고정 (World)



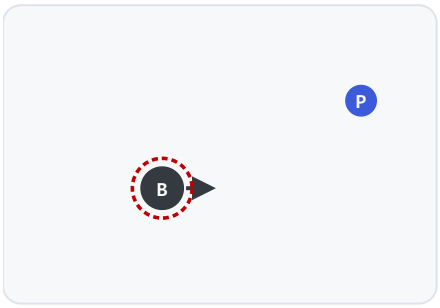
보스·플레이어와 무관, 항상 북쪽(↑)

왜 방향까지 필요한가

1. **방향이 없으면 그릴 수 없는 모양이 있다**
   - 부채꼴 · 일직선 · 투사체 발사 · 돌진 — 전부 "어느 쪽으로"가 있어야 한다
2. **오프셋도 방향 기준이다**
   - "보스 앞 3m" = 보스가 보는 쪽으로 3m. 방향이 없으면 "앞"이 없다
3. **그래서 Anchor = 출처 하나 + 방향 하나 (+ 오프셋)**
   - "기준점 하나면 된다"는 맞다 — 다만 그 한 점이 방향을 같이 들고 있어야 한다

4. 기준점의 출처 3가지

Caster — 시전자



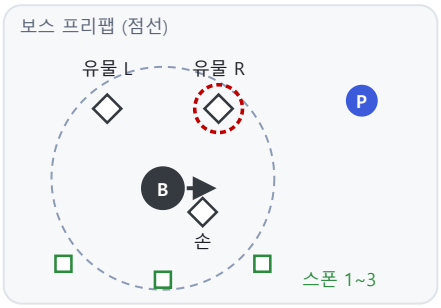
`owner.transform`

Target — 지금 타겟



`targets.Current` (물어볼 때마다)

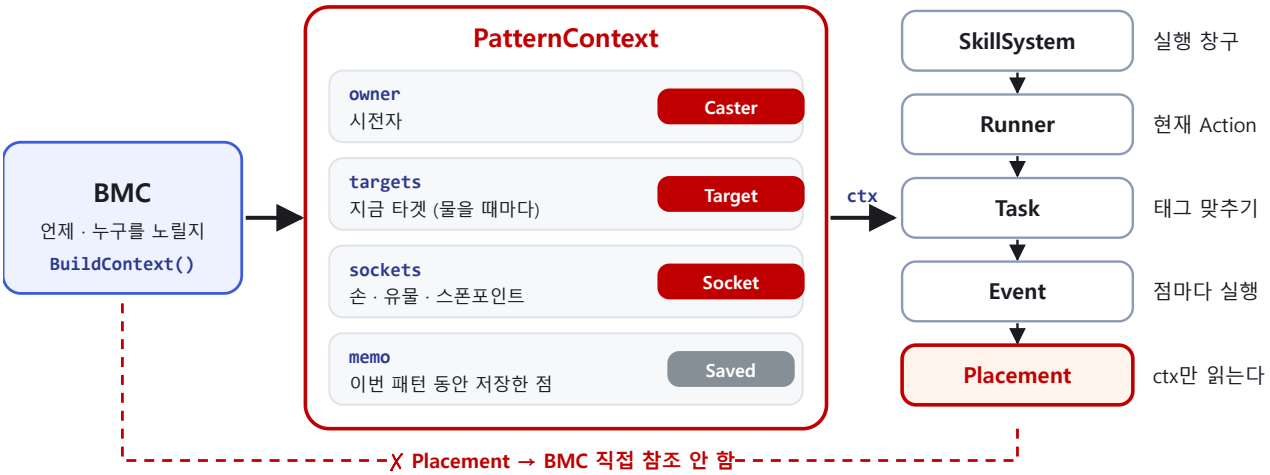
Socket — 이름 붙은 점



`sockets["Relic_R"]`

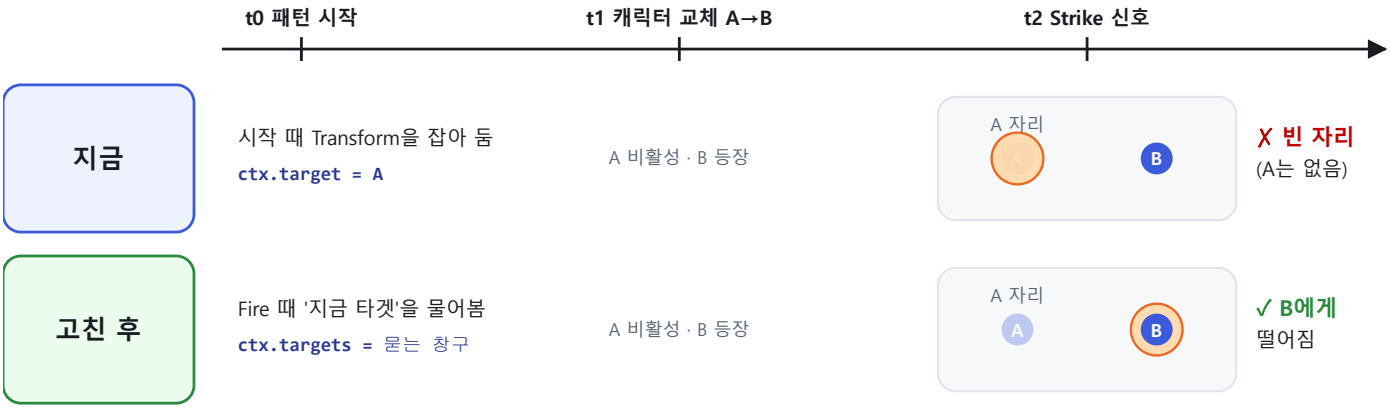
5. 기준점 정보는 어디서 오나 — PatternContext 하나로

BMC가 **BuildContext()**에서 채워 넣고, 아래층은 전부 ctx만 넘겨받는다. Placement는 **BMC가 누구인지 모른다** — 몹 · 보스 · 플레이어 누가 시전해도 같은 Placement가 돈다.



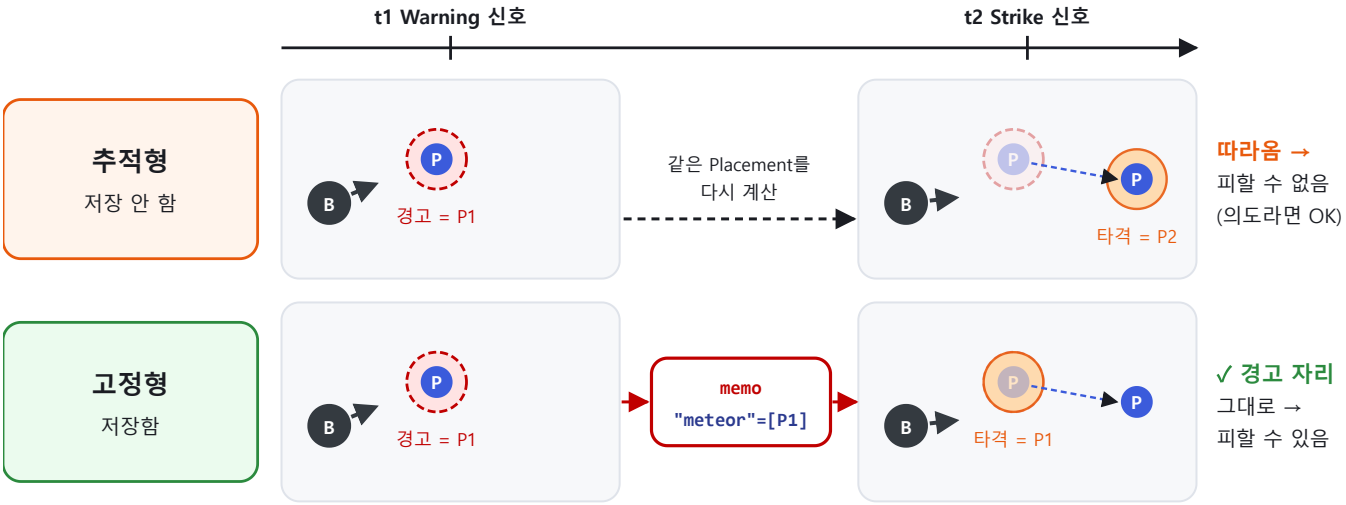
6. 언제 읽나 ① — 타겟은 "Fire 순간"에 물어야 한다

패턴 시작(t0)과 Strike 신호(t2) 사이는 몇 초. 그 사이 파티 캐릭터를 바꾸면?



7. 언제 읽나 ② — 경고와 타격은 다른 신호다

`Event.Warning`과 `Event.Strike`는 다른 프레임에 온다. 그 사이 플레이어가 움직이면 같은 Placement라도 **다른 자리**가 나온다.



	WarningEvent의 Placement	StrikeEvent의 Placement
추적형	타겟 · 점	타겟 · 점 같은 것
고정형	타겟 · 점 + `saveAs = "meteor"`	`SavedPlacement("meteor")`

고정이냐 추적이냐 = 코드가 아니라 에셋 선택

8. 코드 뼈대

```
public enum AnchorSource { Caster, Target, Socket }
public enum FacingSource { AnchorForward, TowardTarget, World }

[Serializable] public struct Anchor
{
    public AnchorSource source;   // 어디서 — 시전자 · 타겟 · 소켓
    public string       key;      // Socket일 때 이름 ("Relic_R")
    public Vector3      offset;   // 방향 기준 오프셋 ("앞 3m" = (0,0,3))
    public FacingSource facing;   // 어느 쪽을 앞으로 볼지

    public bool Resolve(in PatternContext ctx, out Pose pose);   // ctx만 읽는다
}

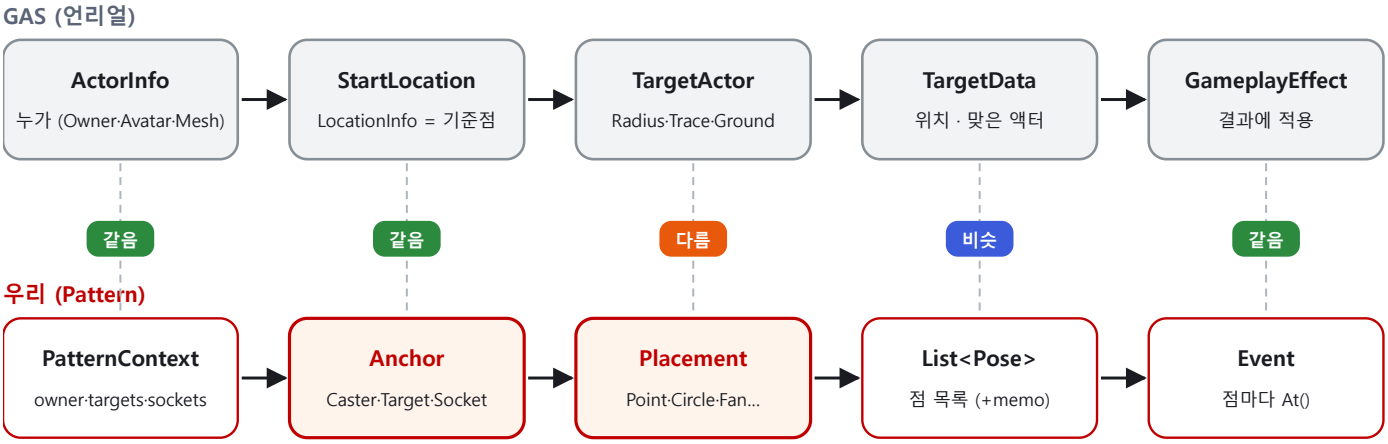
public abstract class Placement : MonoBehaviour
{
    [SerializeField] protected Anchor _anchor;

    public virtual void Collect(in PatternContext ctx, List<Pose> results)
    {
        if (_anchor.Resolve(ctx, out var origin)) Generate(origin, results);
    }
    protected abstract void Generate(Pose origin, List<Pose> results);   // 순수 기하 — 테스트 · 기즈모
}

// 고정형 전용 — 기준점 대신 memo에 저장된 점을 그대로 돌려준다
public class SavedPlacement : Placement { override Collect → ctx.memo[_key] 복사 }
```

9. GAS와 나란히 놓으면

GAS에도 "어디 · 누구"만 맡는 층(**Targeting**)이 따로 있다. 어빌리티가 TargetActor를 띄우고 → TargetActor가 TargetData를 만들고 → 그 결과에 GameplayEffect를 건다.



GAS StartLocation의 LocationType	뜻	우리 Anchor
`ActorTransform`	액터(SourceActor)의 위치 · 방향	**Caster / Target** 같음
`SocketTransform`	메시(SourceComponent)의 소켓 이름	**Socket** — IAnchorSet 키 같음
`LiteralTransform`	값으로 박아 둔 좌표	**SavedPlacement** — memo에 저장된 점 비슷

"기준점 하나 + 출처 선택" = GAS LocationInfo와 같은 결론

GAS와 다른 점 3가지

1. **GAS TargetActor는 "누구"까지 고른다 — 우리 Placement는 "어디"까지만**
   - TargetActor_Radius = 반경 안 액터 목록을 돌려줌 / Placement = 점만, 맞은 대상은 Event(StrikeEvent의 OverlapSphere)가 찾는다
   - 이유: 보스 패턴은 맞을 사람보다 떨어질 자리가 먼저 — 메테오는 빈 땅에도 떨어지고, 경고 데칼은 대상이 없다
2. **조준(확인 · 취소 · 미리보기)은 안 가져왔다**
   - GAS TargetActor는 플레이어가 고르고 확정하는 흐름까지 포함 / 보스는 조준하지 않는다 → Collect는 바로 끝나는 순수 함수
3. **결과를 넘기는 거리가 다르다**
   - GAS: TargetData를 같은 어빌리티 안 다음 단계에 바로 넘김 / 우리: 다른 신호(Warning → Strike) 사이에 넘겨야 해서 PatternMemo에 저장